

The Complete Prefix Sum Roadmap (Beginner → Advanced)

62 hand-picked problems, organized by concept dependency (not difficulty), with verified LeetCode numbers.  = LeetCode Premium (locked unless you subscribe — noted so you're not confused when you can't find it).

Level 0 — Prerequisites (no problems, just make sure you have these)

- Arrays & basic loops
 - Strings & character indexing
 - HashMap / HashSet (get/put/containsKey)
 - Modular arithmetic (%), and why $(a \% m) + m) \% m$ avoids negative remainders
 - Two pointers (fixed idea of `left/right` indices)
 - Basic recursion/DP is *not* required yet — it'll come up only in a few Level 11–12 problems
-

Master Roadmap (by Level → Pattern)

Level 1 — Prefix Sum Fundamentals

Goal: internalize $prefix[i] = prefix[i-1] + nums[i]$ and $sum(l, r) = prefix[r+1] - prefix[l]$.

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
1	1480	Running Sum of 1d Array	Easy	Basic Prefix Sum	Arrays	1
2	724	Find Pivot Index	Easy	Basic Prefix Sum	#1	2
3	303	Range Sum Query - Immutable	Easy	Range Sum Query	#1	3
4	1732	Find the Highest Altitude	Easy	Basic Prefix Sum	#1	4
5	1588	Sum of All Odd Length Subarrays	Easy	Prefix Sum Contribution	#1	5
6	1652	Defuse the Bomb	Easy	Circular Prefix Sum	#1	6
7	2270	Number of Ways to Split Array	Medium	Prefix Sum Split-Point	#1–3	7

Level 2 — Prefix Sum on Arrays (subarray/complement relationships)

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
---	------	--------------	------------	---------	--------------	-------

8	1013	Partition Array Into Three Parts With Equal Sum	Easy	Prefix Sum + Two Pointer	Level 1	8
9	238	Product of Array Except Self	Medium	Prefix/ Suffix Product (sibling concept)	Level 1	9
10	1352	Product of the Last K Numbers	Medium	Prefix Product Design	#9	10

Level 3 — Prefix Sum + HashMap

The single most important idea in this whole roadmap.

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
11	560	Subarray Sum Equals K	Medium	Prefix Sum + HashMap 🔥 CORE	Level 1–2, HashMap	11
12	325	Maximum Size Subarray Sum Equals k 	Medium	Prefix Sum + HashMap	#11	12

13	1546	Max Non-Overlapping Subarrays With Sum Equals Target	Medium	Prefix Sum + Greedy HashMap	#11	13
14	1477	Find Two Non-overlapping Sub-arrays Each With Target Sum	Medium	Prefix Sum + HashMap + DP	#13	14
15	525	Contiguous Array	Medium	Prefix Sum (map 0→-1) + HashMap	#11	15

Level 4 — Prefix Sum + Modulo

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
16	974	Subarray Sums Divisible by K	Medium	Prefix Sum + Modulo 🔥 CORE	Level 3, modulo	16
17	523	Continuous Subarray Sum	Medium	Prefix Sum + Modulo + HashMap	#16	17
18	1590	Make Sum Divisible by P	Medium	Prefix Sum + Modulo + HashMap (harder)	#16–17	18

Level 5 — Prefix Sum + Counting

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
19	930	Binary Subarrays With Sum	Medium	Prefix Sum + Counting (atMost trick)	Level 3—4	19
20	1248	Count Number of Nice Subarrays	Medium	Prefix Sum + Counting (parity)	#19	20
21	1524	Number of Sub-arrays With Odd Sum	Medium	Prefix Sum Parity Counting	#20	21
22	2559	Count Vowel Strings in Ranges	Medium	Prefix Sum Range Counting	Level 1, 3	22

Level 6 — Prefix Sum + Sliding Window / Two Pointers

Where these techniques overlap — and, more importantly, where they diverge.

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
---	------	--------------	------------	---------	--------------	-------

23	209	Minimum Size Subarray Sum	Medium	Sliding Window (contrast case — positives only)	Level 1	23
24	1712	Ways to Split Array Into Three Subarrays	Medium	Prefix Sum + Binary Search/ Two Pointer	Level 1, 10	24
25	862	Shortest Subarray with Sum at Least K	Hard	Prefix Sum + Monotonic Deque 🔥 CORE-HARD	Level 3, deque basics	25

Level 7 — Prefix Sum + Kadane's Algorithm

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
26	53	Maximum Subarray	Medium	Kadane's Algorithm 🔥 CORE	Level 1	26
27	918	Maximum Sum Circular Subarray	Medium	Kadane + Prefix (total – min subarray)	#26	27
28	1749	Maximum Absolute Sum of Any Subarray	Medium	Prefix Sum max – min	#26	28

29	152	Maximum Product Subarray	Medium	Kadane variant (product, not sum)	#26	29
30	1191	K-Concatenation Maximum Sum 	Medium	Kadane + Prefix (repeated array)	#26–27	30

Level 8 — Difference Array

The "inverse" of prefix sum: apply $O(1)$ range updates, then prefix-sum once at the end.

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
31	1094	Car Pooling	Medium	Difference Array  CORE	Level 1	31
32	1109	Corporate Flight Bookings	Medium	Difference Array  CORE	#31	32
33	598	Range Addition II	Easy	Difference Array intuition (2D-lite)	#31	33
34	370	Range Addition 	Medium	Difference Array (textbook)	#31–32	34
35	1854	Maximum Population Year	Easy	Difference Array / Line Sweep	#31	35

36	1893	Check if All the Integers in a Range Are Covered	Easy	Difference Array	#31	36
37	2381	Shifting Letters II	Medium	Difference Array (directional +1/-1)	#31–32	37
38	995	Minimum Number of K Consecutive Bit Flips	Hard	Difference Array + Greedy	#31–32	38
39	2251	Number of Flowers in Full Bloom	Hard	Difference Array/Line Sweep + Binary Search	#31–32, Binary Search	39

Level 9 — 2D Prefix Sum / Matrix Prefix Sum

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
40	304	Range Sum Query 2D - Immutable	Medium	2D Prefix Sum 🔥 CORE	Level 1	40
41	1314	Matrix Block Sum	Medium	2D Prefix Sum	#40	41
42	1292	Maximum Side Length of a Square with Sum $\leq$ Threshold	Medium	2D Prefix Sum + Binary Search	#40	42

43	2536	Increment Submatrix by One	Medium	2D Difference Array	#40, Level 8	43
44	1738	Find Kth Largest XOR Coordinate Value	Medium	2D Prefix XOR	#40	44
45	1074	Number of Submatrices That Sum to Target	Hard	2D Prefix Sum + Subarray Sum = K	#40, #11	45
46	363	Max Sum of Rectangle No Larger Than K	Hard	2D Prefix Sum + Binary Search / Ordered Set	#45	46

Level 10 — Prefix Sum + Binary Search / Sorting

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
47	1292	<i>(cross-ref from Level 9)</i>	Medium	Prefix Sum + Binary Search	—	47
48	363	<i>(cross-ref from Level 9)</i>	Hard	Prefix Sum + Binary Search	—	48
49	1727	Largest Submatrix With Rearrangements	Hard	Column-Height + Sorting + Row Prefix	Level 9, sorting	49

Level 11 — Advanced Prefix Sum (XOR / Bitmask Prefix)

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
50	1310	XOR Queries of a Subarray	Medium	Prefix XOR	Level 1, bitwise XOR	50
51	1442	Count Triplets That Can Form Two Arrays of Equal XOR	Medium	Prefix XOR + Counting	#50	51
52	1371	Longest Substring Containing Vowels in Even Counts	Medium	Prefix Bitmask	#50, bitmasking	52
53	1542	Find Longest Awesome Substring	Hard	Prefix Bitmask + Near-Match	#52	53
54	1177	Can Make Palindrome from Substring 	Medium	Prefix Bitmask	#52	54

Level 11b — Prefix Sum + Monotonic Stack/Deque (contribution technique)

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
---	------	--------------	------------	---------	--------------	-------

55	907	Sum of Subarray Minimums	Medium	Monotonic Stack + Contribution	Monotonic stack basics	55
56	2104	Sum of Subarray Ranges	Medium	Monotonic Stack (max & min contribution)	#55	56
57	1856	Maximum Subarray Min-Product	Medium	Monotonic Stack + Prefix Sum	#55, #26	57
58	862	<i>(cross-ref from Level 6)</i>	Hard	Prefix Sum + Monotonic Deque	—	58

Level 12 — Hard / Interview-Level Capstones

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
59	1074	<i>(cross-ref)</i>	Hard	2D Prefix Sum + HashMap	—	59
60	363	<i>(cross-ref)</i>	Hard	2D Prefix Sum + Binary Search	—	60
61	2251	<i>(cross-ref)</i>	Hard	Difference Array + Binary Search	—	61

62	828	Count Unique Characters of All Substrings of a Given String	Hard	Contribution Technique (prefix-adjacent)	Level 11b	62
----	-----	---	------	--	-----------	----

Recommended Solving Order — The Curriculum

Follow this path problem-by-problem. Skipping is fine once you feel a pattern is automatic — the goal is fluency, not box-checking.

1. LC 1480 – Running Sum of 1d Array. Just build the prefix array. This *is* the whole idea in its rawest form.

2. LC 724 – Find Pivot Index. Teaches `sum(l, r) = prefix[r] - prefix[l]` by forcing you to compare a left-sum to a right-sum at every index.

3. LC 303 – Range Sum Query - Immutable. First "real" use case: precompute once, answer many range queries in $O(1)$. This is *why* prefix sum exists.

4–7. LC 1732, 1588, 1652, 2270. Reinforcement reps — same core idea, slightly different framing (altitude deltas, contribution counting, circular indexing, split points). Don't skip these; they build the pattern-recognition reflex.

8. LC 1013 – Partition Array Into Three Equal Parts. First time you use prefix sum to *search* for split points rather than just answer queries.

9–10. LC 238, 1352. Sibling concept: prefix *product* instead of prefix *sum*. Shows the pattern generalizes beyond addition.

11. LC 560 – Subarray Sum Equals K. STOP AND MASTER THIS

ONE. This is the most important problem in the entire roadmap. It introduces: (a) prefix sum can be negative, so sliding window breaks; (b) a hashmap storing `{prefix_sum: frequency}` turns an $O(n^2)$ brute force into $O(n)$; (c) the algebraic move `prefix[j] - prefix[i] = k` $\rightarrow$ `prefix[i] = prefix[j] - k`. Internalize this move — it recurs in over half of Levels 3–12.

12–15. LC 325, 1546, 1477, 525. Direct variations of #11 — same hashmap idea, different thing being optimized (max length vs. count vs. non-overlap vs. balance-transform). Confirm you can derive each from #11 without looking it up.

16. LC 974 – Subarray Sums Divisible by K. Same hashmap idea as #11, but the key is `prefix_sum % k` instead of the raw sum. This is where you learn *why* the map counts **frequency of prefix remainders**, not just presence.

17–18. LC 523, 1590. Harder modulo variations — length ≥ 2 constraint, and the "find minimum elements to remove" twist.

19–22. LC 930, 1248, 1524, 2559. The counting family: turn "exact match" into `atMost(X) - atMost(X-1)`, or use parity of prefix sums directly.

23. LC 209 – Minimum Size Subarray Sum. Deliberately placed here as a *contrast* case: because all numbers are positive, prefix sum is monotonic, so sliding window (not hashmap) is the right tool. Understanding why 209 is NOT solved like #11 is as important as solving #11 itself.

24. LC 1712. Two-pointer/binary-search over prefix sums for a 3-way split — bridges Levels 6 and 10.

25. LC 862 – Shortest Subarray with Sum at Least K. The hard capstone of prefix sum + sliding window: negatives are allowed (so pure sliding window fails, like #11), but you also need the answer efficiently, so you maintain a **monotonic increasing deque of prefix sums**. This is usually the first "hard" problem people actually struggle with — budget real time for it.

26. LC 53 – Maximum Subarray. Kadane's algorithm, reframed: `dp[i]`

`= max(nums[i], dp[i-1] + nums[i])` is secretly `prefix[i] - min(prefix[0..i-1])`. Solve it both ways to see the connection.

27–30. LC 918, 1749, 152, 1191. Circular arrays, max absolute (not just max) subarray sum, product instead of sum, and repeated-array Kadane. Each bends one assumption of #26.

31–32. LC 1094, 1109 – Difference Array. New skill: instead of *reading* range sums fast, you *write* range updates fast (`diff[l] += v`; `diff[r+1] -= v`), then prefix-sum once at the end. This is the mirror image of Level 1–2.

33–39. Variations and harder difference-array problems (2D-lite, textbook range addition, line sweep for population/coverage, directional shifts, greedy bit-flipping, and the binary-search hybrid in 2251).

40. LC 304 – Range Sum Query 2D - Immutable. Prefix sum extends to 2 dimensions: `prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1] + matrix[i][j]` (inclusion-exclusion). This formula is the single hardest thing to memorize in the whole roadmap — drill it until it's automatic.

41–46. Direct 2D applications: block sums, thresholded square search (+ binary search), 2D difference arrays, 2D XOR prefix, and the two hardest 2D problems (1074, 363) which combine 2D prefix sum with the Level 3 hashmap trick.

47–49. Binary-search-flavored prefix sum problems, capped by 1727 (sort + prefix, genuinely advanced).

50–54. Prefix sum under XOR and under bitmask — same `prefix[i] - prefix[j]` idea, but with XOR (self-canceling) or a bitmask (parity of 26 letters) replacing addition.

55–58. Monotonic stack "contribution" problems: instead of prefix sums of values, you sum each element's *contribution* across all subarrays where it's the min/max — a distinct but related mental model.

59–62. Final capstones combining everything: 2D + hashmap, 2D + binary search, difference array + binary search, and substring contribution counting.

🔥 Must-Do vs ☆ Recommended vs 💡 Optional vs 🏆 Hard Challenge

🔥 **Must-Do (18)** — non-negotiable, appear constantly in interviews
1480, 724, 303, 560, 325, 974, 209, 53, 918, 1094, 1109, 304, 1074, 862, 930, 1248, 523, 1310

☆ **Recommended (24)** — strong variations that cement the pattern
1732, 2270, 1013, 238, 1546, 1477, 525, 1590, 1524, 2559, 1712, 1749, 1314, 1292, 2536, 1738, 1442, 1371, 1854, 1893, 2381, 1352, 152, 907

💡 **Optional (11)** — repetitive or narrow; solve only for full mastery
1588, 1652, 598, 370, 995, 1191, 2104, 1856, 1442 (*if already done under Recommended, skip*), 1177, 2559 (*if time-limited*)

🏆 **Hard Challenge (9)** — combine 3+ techniques; do these last
1074, 363, 2251, 1727, 1542, 828, 1856, 995, 2536

(A few problems appear in two buckets above because their difficulty depends on which sub-skill you're weakest in — use your own judgment once you've done the Must-Do set.)

Prefix Sum Cheat Sheet

CORE FORMULAS

```
prefix[0] = 0
```

```
prefix[i] = prefix[i-1] + nums[i-1] (1-indexed pre
```

```
sum(l, r) = prefix[r+1] - prefix[l] (inclusive ra
```

```
For "subarray sum == k":
```

```
    prefix[j] - prefix[i] = k
```

```
    => prefix[i] = prefix[j] - k
```

```
    => at index j, look up how many earlier prefixes equal
```

2D INCLUSION-EXCLUSION

```
prefix[i][j] = prefix[i-1][j] + prefix[i][j-1] - prefix[i-1][j-1]
rectSum(r1,c1,r2,c2) = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]
```

Basic Prefix Sum

- Recognize: "sum of first i elements", "cumulative", repeated range-sum reads on a static array.
- Use when: array doesn't change; you need range sums repeatedly.
- Core idea: precompute once, answer in $O(1)$.
- Time: $O(n)$ build, $O(1)$ query. Space: $O(n)$.

```
prefix = [0]
for x in nums: prefix.append(prefix[-1] + x)
```

Range Sum

- Recognize: "range sum query", multiple (l, r) queries.
- Same as above — this *is* Basic Prefix Sum applied to a query workload.

Prefix Sum + HashMap

- Recognize: "number of subarrays", "subarray sum equals target", array has negatives (rules out sliding window).
- Use when: you need to count/find pairs of prefix sums differing by exactly k .
- Core idea: as you scan, store `{prefix_sum: count_of_times_seen}`; at each index look up `prefix - k`.
- Time: $O(n)$. Space: $O(n)$.

```
seen = {0: 1}
prefix = 0
count = 0
for x in nums:
    prefix += x
    count += seen.get(prefix - k, 0)
```

```
seen[prefix] = seen.get(prefix, 0) + 1
```

Subarray Sum = K

- This is the archetype of the pattern above — see LC 560. Every "count subarrays with property X" problem is a reskin of this template with a different key (raw sum, `sum % k`, parity, bitmask, etc).

Prefix Sum + Modulo

- Recognize: "divisible by k", "`sum % k == 0`".
- Use when: you care about remainder equality, not exact sum equality.
- Core idea: two prefix sums with the same remainder mod k mean the subarray between them is divisible by k.
- Formula: `((prefix[j] % k) - (prefix[i] % k)) % k == 0` $\Leftrightarrow$ `prefix[j] % k == prefix[i] % k`
- Time: $O(n)$. Space: $O(k)$.

```
seen = {0: 1}    # remainder -> count
prefix = 0
count = 0
for x in nums:
    prefix = (prefix + x) % k
    prefix = (prefix + k) % k    # normalize negative remainder
    count += seen.get(prefix, 0)
    seen[prefix] = seen.get(prefix, 0) + 1
```

Counting Subarrays (atMost trick)

- Recognize: "exactly K", "at least K" of some property.
- Core idea: `exactly(K) = atMost(K) - atMost(K-1)`.
Often paired with sliding window when values are non-negative, or with the HashMap template when they aren't.

Kadane's Algorithm

- Recognize: "maximum subarray sum", "maximum/minimum contiguous sum".

- Use when: no counting or indices needed, just the optimum value (or its bounds).
- Core idea: `best_ending_here = max(x, best_ending_here + x)`; track running max. Equivalent to `prefix[i] - min(prefix[0..i-1])`.
- Time: $O(n)$. Space: $O(1)$.

```
best = cur = nums[0]
for x in nums[1:]:
    cur = max(x, cur + x)
    best = max(best, cur)
```

Difference Array

- Recognize: "range update", "add v to all elements from l to r ", "multiple bookings/flights/bookings over ranges".
- Use when: many range *updates* need to be applied efficiently, and you only need the final array (not intermediate states).
- Core idea: mark the start ($+v$) and just-past-the-end ($-v$) of each range; prefix-sum the diff array once at the end.
- Time: $O(n + q)$. Space: $O(n)$.

```
diff = [0] * (n + 1)
for (l, r, v) in updates:
    diff[l] += v
    diff[r + 1] -= v
result = []
running = 0
for d in diff[:-1]:
    running += d
    result.append(running)
```

2D Prefix Sum

- Recognize: "sum of a submatrix/rectangle", multiple 2D range queries.
- Use when: matrix is static, and you need repeated rectangle sums.
- Core idea: inclusion-exclusion (see formula block above).

- Time: $O(mn)$ build, $O(1)$ query. Space: $O(mn)$.

Prefix Sum + Binary Search

- Recognize: "smallest/largest range such that $\text{sum} \leq/\geq X$ ", monotonic prefix array (all non-negative), optimization over a threshold.
- Use when: prefix array is sorted/monotonic, so you can binary-search for a boundary instead of scanning.
- Time: $O(n \log n)$.

Why does the HashMap store *frequency* of prefix sums (not just existence)? Because the same prefix-sum value can occur at multiple earlier indices, and *each* earlier occurrence defines a distinct valid subarray ending at the current index. If you only stored a boolean/last-index, you'd undercount (for "how many subarrays") or you'd need the *first* index specifically (for "longest subarray") — the map's job depends on which of those two you need, so always ask "am I counting subarrays, or finding an extremal length?" before deciding whether to store frequency or first-index.

How to Identify a Prefix Sum Problem in an Interview

Common phrase clues:

- "sum of a subarray" / "range sum" / "sum between indices l and r"
- "number of subarrays with sum ..." / "subarray sum equals K"
- "sum divisible by K"
- "multiple range update operations" (→ Difference Array, not plain Prefix Sum)
- "rectangle sum" / "submatrix sum" (→ 2D Prefix Sum)
- "maximum/minimum subarray sum" (→ Kadane's, which is prefix sum in disguise)
- Repeated range queries on a **static** array/matrix

How to tell Prefix Sum apart from lookalikes:

If the problem...	Use...
Has only non-negative numbers and asks for shortest/longest window meeting a sum condition	Sliding Window (prefix sum is monotonic here, so a window works and is simpler)
Has negative numbers and asks for count/existence of a target-sum subarray	Prefix Sum + HashMap (window breaks because sums aren't monotonic)
Asks only for the <i>single best</i> contiguous sum (not count, not indices)	Kadane's Algorithm (no need for a full prefix array or hashmap)
Involves many <i>range update</i> operations, then a final read	Difference Array , not prefix sum for reading
Compares two sorted pointers converging inward (pair sums, palindrome checks)	Two Pointers — unrelated to prefix sum unless sums are involved
Needs a fast search for a threshold/boundary over an already-computed (monotonic) prefix array	Prefix Sum + Binary Search

Rule of thumb: if you find yourself about to write a nested loop to recompute a range sum or range update repeatedly, that's the signal — stop and ask "can I precompute this once?"

Beginner Solving Strategy — Step by Step

1. **Understand the brute-force solution first.** Write the $O(n^2)$ or $O(n^3)$ version even if you know it'll be slow — it clarifies exactly what's being repeated.
2. **Identify whether repeated range/subarray calculations exist.** Are you recomputing the same sum/product/XOR multiple times across the algorithm?
3. **Ask whether a prefix representation eliminates that repeated work.** If yes, that's your green light.

4. **Construct the prefix array** (1D or 2D, sum/product/XOR/bitmask as appropriate).
 5. **Derive the formula you need** — usually `prefix[j] - prefix[i] = target`, solved for the unknown side.
 6. **Determine whether a HashMap is required.** Ask: am I just reading ranges (no map needed), or searching for a matching earlier prefix (map needed)?
 7. **Determine what the HashMap should store** — frequency (for counting) vs. earliest index (for max length) vs. latest index (for min length in some variants).
 8. **Check whether modulo is involved.** If "divisible by k" appears, switch your map key from raw prefix sum to `prefix % k`, and normalize negative remainders.
 9. **Check whether Sliding Window, Two Pointers, or Kadane's is actually a better/simpler fit** — see the comparison table above. Don't force prefix sum where a simpler tool applies.
 10. **Implement the solution.**
 11. **Analyze time and space complexity** explicitly — say it out loud, don't just assume $O(n)$.
 12. **Re-solve from scratch without looking at your old solution** a day or two later. This is the step people skip, and it's the one that actually builds retention.
-

Brute Force → Optimized Progression (worked examples)

LC 560 — Subarray Sum Equals K

Brute Force: try every (i, j) pair, sum the slice

↓

Notice: `sum(i, j)` is recomputed from scratch every time

↓

Prefix Sum: precompute `prefix[]`, `sum(i, j) = prefix[j] - prefix[i]`

↓

Prefix Sum + HashMap: instead of checking all earlier i for `sum(i, j) = k`, look up "have I seen `prefix[j] - k` before?" in $O(1)$

LC 304 — Range Sum Query 2D

Brute Force: for each query, sum every cell in the rectangle

↓

Notice: overlapping rectangles recompute the same cell sum

↓

1D Prefix Sum per row: sum each row's segment in $O(1)$

↓

2D Prefix Sum (inclusion-exclusion): $O(1)$ per query after

LC 862 — Shortest Subarray with Sum at Least K

Brute Force: try every subarray, sum it, check $\geq K$

↓

Notice: negatives mean sliding window can't shrink safely

↓

Prefix Sum: $\text{sum}(i, j) = \text{prefix}[j] - \text{prefix}[i]$ in $O(1)$

↓

Prefix Sum + Monotonic Deque: keep only "useful" earlier
(pop ones that are $\geq$ current — they're strictly worse)

LC 995 — Minimum K Consecutive Bit Flips

Brute Force: simulate flipping a window of size K at every

↓

Notice: each flip is a uniform range update — no need to

↓

Difference Array: mark flip start/end in $O(1)$, track runs

Why prefix sum works, in one sentence: **it converts a range operation (sum, or an update applied to a range) into two point operations (a subtraction of two prefix values, or two array writes), trading $O(\text{range length})$ for $O(1)$.**

Revision Strategy

Stage	When	What to do
First attempt	Day 0	Solve fresh, using the 12-step beginner strategy. Time yourself.
Review 1	Day 1	Re-read your solution. Explain out loud <i>why</i> prefix sum applies. No re-coding needed unless you're shaky.
Review 2	Day 3	Re-solve from scratch, no notes. If stuck >10 min, peek at just the formula, not the code.
Review 3	Day 7	Pattern-recognition only: read the problem statement, state which pattern it is and the formula you'd use, without writing code.
Review 4	Day 14	Re-solve Must-Do problems from scratch, full code, timed.
Final review	Day 30	Do a mixed set: 5 random Must-Do + 3 random Hard Challenge problems back-to-back, no hints, simulating interview pressure.

Classification for revision purposes:

- **Must re-solve from scratch:** all 18 Must-Do problems, every review cycle.
- **Review approach only (no re-coding after Day 3):** most ☆ Recommended problems once solved correctly twice.
- **Pattern-recognition revision only:** 🧠 Optional problems — you just need to recognize them if they reappear, not re-derive them.
- **Optional revision (skip if short on time):** 🏆 Hard Challenges, once solved once correctly — revisit only if targeting top-tier interviews (Google/Meta/Bloomberg-style).

Final Summary

Total curated problems: 62 (plus cross-references, so 62 unique LeetCode problems total)

Difficulty	Count
Easy	12
Medium	38
Hard	12

Pattern	# of Problems
Basic / Range Prefix Sum	7
Prefix Sum on Arrays	3
Prefix Sum + HashMap	5
Prefix Sum + Modulo	3
Prefix Sum + Counting	4
Prefix Sum + Sliding Window/Two Pointers	3
Prefix Sum + Kadane's	5
Difference Array	9
2D Prefix Sum	7
Prefix Sum + Binary Search/Sorting	3 (2 cross-ref)
Advanced (XOR/Bitmask Prefix)	5
Prefix Sum + Monotonic Stack/Deque	4 (1 cross-ref)
Hard Capstones	4 (3 cross-ref)

Bucket	Count
🔥 Must-Do	18
☆ Recommended	24
☁ Optional	11
🏆 Hard Challenge	9

(Note: some problems appear in more than one bucket/category depending on the skill they reinforce — the "62 unique problems" figure

counts each LeetCode problem once.)

If You Have Limited Time

10 problems — Basic Understanding 1480, 724, 303, 560, 974, 209, 53, 1094, 1109, 304

20 problems — Interview Preparation *(the 10 above, plus:)* 325, 918, 1546, 523, 930, 1248, 862, 1013, 238, 1732

30 problems — Strong Mastery *(the 20 above, plus:)* 1477, 525, 1590, 1524, 2559, 1712, 1749, 1314, 1292, 1738

Full 62-problem collection — Complete Mastery Everything in the Master Roadmap above, done in the listed order.

A closing note on accuracy: every problem number and title above was verified against LeetCode's own problem descriptions at the time of writing. Premium-only problems are flagged with  — if you don't have LeetCode Premium, treat those as read-the-editorial-and-understand-the-approach rather than solve-and-submit.